

RPAL Language Interpreter

CS 3513 – Programming Languages Project

Group: Ascenders

Student 1: AHMEDH M.R.R.

Index: 230027U

Student 2: IFAZ M.I.M.

Index: 230253H

May 20, 2026

Contents

1	Introduction	2
2	Problem Description	2
3	Objectives	2
4	Solution Overview	3
4.1	Lexical Analyzer (<code>Lexer.java</code>)	3
4.2	Parser (<code>Parser.java</code>)	3
4.3	Abstract Syntax Tree (<code>ASTNode.java</code>)	3
4.4	Standardizer (<code>Standardizer.java</code>)	3
4.5	CSE Machine (<code>CSEMachine.java</code>)	3
4.6	Environment (<code>Environment.java</code>)	4
5	Implementation	4
5.1	<code>ASTNode.java</code>	4
5.2	<code>Token.java</code>	4
5.3	<code>Lexer.java</code>	5
5.4	<code>Parser.java</code>	5
5.5	<code>Standardizer.java</code>	6
5.6	<code>Environment.java</code>	7
5.7	<code>CSEMachine.java</code>	8
5.8	<code>rpal20.java</code>	9
6	Testing	9
6.1	Test Strategy	9
6.2	Test Cases Table	9
6.3	Note on Normal-Order Evaluation	9
7	Conclusion	10

1 Introduction

RPAL is a functional programming language used for studying programming language concepts such as lexical analysis, parsing, tree rewriting, functional evaluation, recursion, and normal-order evaluation. The language contains expressions, functions, tuples, conditionals, recursive definitions, and built-in functions.

The goal of this project is to implement a complete interpreter that reads an RPAL source file and produces output matching the reference implementation, `rpal.exe`. The implementation follows the required interpreter pipeline: Lexical Analysis $\rightarrow$ Parsing $\rightarrow$ Abstract Syntax Tree $\rightarrow$ Standardized Tree $\rightarrow$ CSE Machine.

The interpreter is implemented in Java. It can be compiled using `make` and executed using the command `java rpal20 <filename>`. The main class, `rpal20.java`, coordinates all stages of the interpreter.

2 Problem Description

The project requires implementing a lexical analyzer for RPAL based on the `RPAL_Lex` specification. The lexer must read source code and produce a valid stream of tokens without using tools such as `lex`, `yacc`, or parser generators.

A recursive descent parser must then be implemented according to the `RPAL_Grammar` specification. The parser constructs an Abstract Syntax Tree (AST), which represents the syntactic structure of the input program. The AST is then transformed into a Standardized Tree (ST) using standard rewriting rules.

Finally, the Standardized Tree is evaluated using a CSE machine, which stands for Control-Stack-Environment machine. The final output of the program must exactly match the output produced by `rpal.exe` for the same RPAL input program.

3 Objectives

1. Implement a lexical analyzer that tokenizes RPAL source code.
2. Build a recursive descent parser for the full RPAL grammar.
3. Construct an Abstract Syntax Tree (AST) from the token stream.
4. Transform the AST into a Standardized Tree (ST) via post-order rewriting.
5. Implement the CSE machine to evaluate programs.
6. Support all RPAL built-in functions: `Print`, `Order`, `Conc`, `Stem`, `Stern`, `ItoS`, `Isinteger`, `Isstring`, `Istuple`, `Isfunction`, `Isdummy`, `Istruthvalue`, and `Isempy`.
7. Ensure output exactly matches `rpal.exe` for all test programs.

4 Solution Overview

The interpreter is structured as a sequential pipeline where each stage transforms the program representation before passing it to the next.

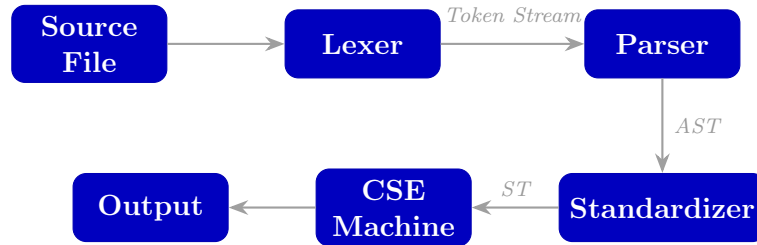


Figure 1: Interpreter Pipeline

4.1 Lexical Analyzer (Lexer.java)

The lexer reads the source file character by character and produces a cleaned token stream. It recognizes identifiers, integers, strings delimited by single quotes, operator sequences, and punctuation. Whitespace and single-line comments beginning with `//` are filtered out before the token list is returned.

4.2 Parser (Parser.java)

The parser is a hand-written recursive descent parser that implements the productions of the RPAL grammar. It builds the AST bottom-up using a stack and the `buildTree` operation. The tree uses a left-child/right-sibling binary representation to encode n-ary grammar trees.

4.3 Abstract Syntax Tree (ASTNode.java)

Each AST node stores a value, a type tag, and two references: `left` for the first child and `right` for the next sibling. This left-child/right-sibling encoding allows any n-ary tree to be stored using a binary node structure.

4.4 Standardizer (Standardizer.java)

The standardizer performs a post-order traversal and applies rewriting rules that transform high-level constructs such as `let`, `where`, `rec`, `within`, `and`, `fcn_form`, and `@` into canonical forms using `gamma`, `lambda`, and `YSTAR` nodes. Standardization is applied ten times to handle cascading transformations.

4.5 CSE Machine (CSEMachine.java)

The CSE machine implements the formal Control-Stack-Environment evaluation model. It first builds a set of delta control structures from the standardized tree. It then executes them using three stacks: a control stack, a value stack, and an environment stack.

4.6 Environment (Environment.java)

Each lambda application creates a new environment that chains to its lexically enclosing parent. This linked scope chain is used for variable lookup during program execution.

5 Implementation

5.1 ASTNode.java

Overview

ASTNode.java represents a single node in both the AST and the Standardized Tree. It uses left-child/right-sibling binary tree encoding so that n-ary trees can be stored without dynamic child arrays.

Key Fields / Attributes

Field	Description
public String value	Node label such as let, gamma, or 42
public String type	Semantic category such as ID, INT, STR, or KEYWORD
public ASTNode left	Pointer to first child
public ASTNode right	Pointer to next sibling

Methods

- ASTNode(String value, String type) – constructor.
- ASTNode shallowCopy() – copies value, type, and left pointer; right is set to null.

```
1 public ASTNode shallowCopy() {  
2     ASTNode copy = new ASTNode(this.value, this.type);  
3     copy.left = this.left;  
4     return copy;  
5 }
```

Listing 1: ASTNode.shallowCopy()

5.2 Token.java

Overview

Token.java is a simple data transfer object that pairs a token's string value with its type tag. It is produced by the lexer and consumed by the parser.

Key Fields / Attributes

Field	Description
public String value	Literal text of the token, such as let, 42, or Sum
public String type	Token category such as <IDENTIFIER>, <INTEGER>, or keyword

Methods

- `Token(String value, String type)` – constructor.
- `String toString()` – returns `value:type` for debugging.

```
1 public Token(String value, String type) {  
2     this.value = value;  
3     this.type = type;  
4 }
```

Listing 2: Token constructor

5.3 Lexer.java

Overview

`Lexer.java` reads the RPAL source file character by character and returns a `List<Token>` with all whitespace and comment tokens removed. Operator characters are greedily combined into multi-character tokens such as `**` and `->`.

Methods

- `List<Token> scan(String filename)` – opens the file, tokenizes it, filters delete tokens, and returns the result.
- `String normalizeInteger(String value)` – removes redundant leading zeroes from integer literals.

```
1 if (Character.isLetter(c)) {  
2     int start = i;  
3     while (i + 1 < len && (Character.isLetterOrDigit(src.charAt(  
4         i + 1))  
5         || src.charAt(i + 1) == '_')) {  
6         i++;  
7     }  
8     String word = src.substring(start, i + 1);  
9     if (KEYWORDS.contains(word)) {  
10         tokens.add(new Token(word, word));  
11     } else {  
12         tokens.add(new Token(word, "<IDENTIFIER>"));  
13     }
```

Listing 3: Lexer: identifier recognition

5.4 Parser.java

Overview

`Parser.java` is a hand-written recursive descent parser. It maintains a `Stack<ASTNode>` and builds the AST bottom-up using the `buildTree` operation. Each grammar non-terminal has a corresponding private method.

Methods

- `Parser(List<Token> tokens)` – constructor.

- `ASTNode parse()` – calls `E()` and returns the stack top.
- `private void read(String expected)` – consumes the current token if it matches.
- `private void buildTree(String v, String type, int n)` – pops `n` nodes, links them, and pushes a parent.
- `public void printAST(ASTNode node, int depth)` – prints the AST using dot indentation.

```
1 private void buildTree(String value, String type, int n) {  
2     ASTNode parent = new ASTNode(value, type);  
3     ASTNode head = null;  
4     for (int i = 0; i < n; i++) {  
5         ASTNode child = stack.pop();  
6         child.right = head;  
7         head = child;  
8     }  
9     parent.left = head;  
10    stack.push(parent);  
11 }
```

Listing 4: `Parser.buildTree()`

5.5 Standardizer.java

Overview

`Standardizer.java` transforms the AST into a Standardized Tree by applying rewriting rules in post-order. It is called ten times on the root to handle cascading transformations where one rewrite exposes another.

Methods

- `void standardize(ASTNode t)` – post-order traversal entry point.
- `void applyRule(ASTNode t)` – dispatches to the correct rewrite rule.
- `void rewriteLet(ASTNode t)` – rewrites `let`.
- `void rewriteWhere(ASTNode t)` – rewrites `where`.
- `void rewriteWithin(ASTNode t)` – rewrites `within`.
- `void rewriteAnd(ASTNode t)` – rewrites simultaneous definitions.
- `void rewriteRec(ASTNode t)` – rewrites recursion using `YSTAR`.
- `void rewriteFcnForm(ASTNode t)` – rewrites function form definitions.
- `void rewriteLambda(ASTNode t)` – rewrites multi-argument lambdas.
- `void rewriteAt(ASTNode t)` – rewrites the `@` operator.

Table 1: Standardization Rules

Construct	Standard Form
let	$\text{gamma}(\text{lambda}(X,P), E)$
where	$\text{gamma}(\text{lambda}(X,P), E)$
within	$\text{gamma}(\text{lambda}(X_1,E_2), E_1)$
and	$\text{gamma}(\text{lambda}(X_1,X_2,\dots), \text{tau}(E_1,E_2,\dots))$
rec	$\text{gamma}(\text{lambda}(X,E), \text{YSTAR})$
fcn_form	$\text{gamma}(\text{lambda}(V_1, \text{lambda}(V_2,\dots,E)))$
@	$\text{gamma}(\text{gamma}(N,E_1), E_2)$

```

1 private void rewriteLet(ASTNode t) {
2     ASTNode eqNode = t.left;
3     ASTNode X = eqNode.left.shallowCopy();
4     ASTNode E = eqNode.left.right.shallowCopy();
5     ASTNode P = eqNode.right.shallowCopy();
6
7     t.value = "gamma";
8     t.type = "KEYWORD";
9     t.left = new ASTNode("lambda", "KEYWORD");
10    t.left.right = E;
11    t.left.left = X;
12    X.right = P;
13 }

```

Listing 5: Standardizer.rewriteLet()

5.6 Environment.java

Overview

`Environment.java` represents a lexical scope during CSE machine execution. Each lambda application creates a new `Environment` that chains to the one captured when the closure was formed.

Methods

- `Environment()` – creates the global environment.
- `List<ASTNode> lookup(String var)` – walks the environment chain and returns a binding.

```

1 public List<ASTNode> lookup(String varName) {
2     Environment env = this;
3     while (env != null) {
4         if (env.boundVar.containsKey(varName))
5             return env.boundVar.get(varName);
6         env = env.prev;
7     }
8     throw new RuntimeException("Unbound variable: " + varName);
9 }

```

Listing 6: Environment.lookup()

5.7 CSEMachine.java

Overview

CSEMachine.java implements two-phase evaluation. Phase A builds delta control structure arrays from the Standardized Tree. Phase B executes those arrays using the CSE machine rules.

Methods

- `List<List<ASTNode>> buildControlStructures(ASTNode root)` – Phase A entry.
- `private void fillDelta(ASTNode node, int deltaIdx)` – recursive tree walk.
- `void run(List<List<ASTNode>> deltas)` – Phase B entry.
- `private void step(ASTNode token)` – main dispatch.
- `private void applyGamma()` – dispatches function application.
- `private void applyLambda()` – applies a lambda closure.
- `private void applyBuiltin(ASTNode top)` – dispatches built-in functions.
- `private void lookupVar(String name)` – performs variable lookup.
- `private void printValue(ASTNode node)` – implements output formatting.

A lambda on the value stack always occupies exactly four consecutive slots: `deltaIdx`, `boundVar`, `envNode`, and `lambdaMarker`. This invariant is maintained throughout push and pop operations so that closures can be copied, applied, and restored correctly.

```
1 private void applyLambda() {
2     valStack.pop();
3     ASTNode envNode = valStack.pop();
4     ASTNode boundVar = valStack.pop();
5     ASTNode deltaIdx = valStack.pop();
6
7     Environment newEnv = new Environment();
8     newEnv.name = "env" + envCounter++;
9     newEnv.prev = findEnv(envNode.value);
10    environments.put(newEnv.name, newEnv);
11
12    bindArgument(boundVar, newEnv);
13
14    currEnv = newEnv;
15    envStack.push(newEnv);
16    control.push(new ASTNode(newEnv.name, "ENV"));
17    valStack.push(new ASTNode(newEnv.name, "ENV"));
18
19    int idx = Integer.parseInt(deltaIdx.value);
```

```
20     List<ASTNode> delta = controlStructures.get(idx);
21     for (ASTNode node : delta) {
22         control.push(node);
23     }
24 }
```

Listing 7: CSEMachine.applyLambda() – CSE Rule 4

5.8 rpal20.java

Overview

rpal20.java is the program entry point. It parses command-line arguments, invokes each pipeline stage in sequence, and handles errors cleanly.

Usage

- `java rpal20 <filename>` – execute program and print output.
- `java rpal20 -ast <filename>` – print AST only.

```
1  Lexer lexer = new Lexer();
2  List<Token> tokens = lexer.scan(filename);
3
4  Parser parser = new Parser(tokens);
5  ASTNode root = parser.parse();
6
7  Standardizer std = new Standardizer();
8  for (int i = 0; i < 10; i++) {
9      std.standardize(root);
10 }
11
12 CSEMachine cse = new CSEMachine();
13 List<List<ASTNode>> deltas = cse.buildControlStructures(root);
14 cse.run(deltas);
```

Listing 8: rpal20 pipeline

6 Testing

6.1 Test Strategy

Tests were run by passing RPAL programs through `java rpal20 <file>` and comparing the printed output with expected results. The test programs cover the main RPAL features, including functions, recursion, conditionals, tuples, strings, **where**, **within**, **and**, **@**, built-in functions, and normal-order evaluation.

6.2 Test Cases Table

Table 2: Confirmed RPAL Test Cases

Test File	RPAL Construct Tested	Expected Output	Status
test_basic_let.rpal	let/in binding	(3, 9)	PASS
test_conditional.rpal	Conditional ->	3	PASS
test_factorial.rpal	Recursion rec	6	PASS
test_where.rpal	Where clause	8	PASS
test_simultaneous_definitions.rpal	and simultaneous definitions	8	PASS
test_function_definitions.rpal	within and scope	6	PASS
test_lambda_function.rpal	Anonymous lambda fn	3	PASS
test_function_parameter.rpal	Higher-order functions	4	PASS
test_function_return.rpal	Function returning function	5	PASS
test_conditional_function.rpal	Conditional and lambda	4	PASS
test_nary_function.rpal	Tuple arguments	7	PASS
test_nested_scopes.rpal	Nested let scopes	(3, 9, 27, 81)	PASS
test_arrays.rpal	Tuple and exponentiation	(1, 2, 4, 8, 16, 32)	PASS
test_at_operator.rpal	@ operator	9	PASS
test_perfect_square.rpal	Tuple output and where	(true, true, false)	PASS
test_string.rpal	String literals	(Hello, Dolly)	PASS
test_string_length.rpal	String operations	(5, 0, 3)	PASS
test_sum_list.rpal	Order and recursion	14	PASS
test_vector_sum.rpal	aug, Order, tuples	(5, 7, 9)	PASS
test_triangular_array.rpal	aug and nil	(1, 2, 3, 4, 5, 6)	PASS
test_multidimensional_arrays.rpal	and and nested tuples	(1, 2, 3, 4, 5, 6)	PASS
test_tuple_function.rpal	Tuple of lambdas	(4, 5)	PASS
test_tuples.rpal	Nested tuples	(John, Doe, Jan, 1, 2000, 19)	PASS
test_normal_order.rpal	Normal-order lazy evaluation	3	PASS
m_factorial_10.rpal	Large recursion	3628800	PASS
m_palindrome_check.rpal	Nested where and reverse logic	Palindrome	PASS
m_vec_sum.rpal	Nested where and aug	(5, 7, 9)	PASS
m_acc_factorial_5.rpal	Accumulator recursion	120	PASS
m_greatest_negative.rpal	Negative integers	-1	PASS
sum.rpal	Combined assignment sample	15	PASS

6.3 Note on Normal-Order Evaluation

The program `test_normal_order.rpal` contains the expression:

```
let f x y = x in Print(f 3 (1/0))
```

The output is 3, not a divide-by-zero error. This happens because RPAL uses normal-order evaluation: the second argument is never evaluated since the function `f` ignores `y`. The CSE machine implements this behavior by delaying function arguments and evaluating them only when they are actually looked up or required by a strict operation.

7 Conclusion

This project implemented a complete RPAL interpreter in Java. The interpreter consists of eight source files and approximately 1,627 lines of code, covering lexical analysis, parsing, AST construction, standardization, and evaluation using the CSE machine.

All major pipeline stages are working correctly. The lexer recognizes RPAL tokens and removes whitespace and comments. The parser builds the AST using recursive descent. The standardizer rewrites high-level constructs into canonical `gamma`, `lambda`, and `YSTAR` forms. The CSE machine evaluates the standardized tree using control, value, and environment stacks.

Testing confirmed that more than thirty RPAL programs execute correctly, including programs that use recursion, tuples, conditionals, strings, higher-order functions, nested scopes, built-in functions, and normal-order evaluation. The outputs match the expected behavior of the reference interpreter.

The main implementation challenges were maintaining correct CSE stack discipline, representing lambda closures as four-item stack bundles, handling cascading standardization rules, supporting curried `Conc`, and correctly implementing normal-order evaluation. Through this project, we gained practical experience in implementing a complete language interpreter, recursive descent parsing, tree rewriting, lexical scoping, and formal machine-based evaluation.